Project - Low Level Design

on

Chat with Retail Documents (RAG)

Course Name:GenAi

Institution Name: Medi caps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Aditya Agrawal	EN22IT301006
2.	Atharv Shukla	EN22IT301025
3.	Malya Singh	EN24CA5030092
4.	Mohit Gupta	EN21CA301036
5.	Nikhil Kumar Verma	EN23IT3L1002

Group Name: Group 03D12

Project Number: DO-41

Industry Mentor Name: Mr. Yash Bansal

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2026

Table of Contents

1. **Introduction**
   1.1 Scope of the Document
   1.2 Intended Audience
   1.3 System Overview

2. **System Design**
   2.1 Application Design
   2.2 Process Flow
   2.3 Information Flow
   2.4 Components Design
   2.5 Key Design Considerations
   2.6 API Catalogue

3. **System Architecture Diagram**

4. **RAG Pipeline Diagram**

5. **Sequence Diagram**

6. **Technology Stack**

7. **References**

1. Introduction

The **Retail Document Chatbot** is an intelligent question-answering system that allows users to interact with retail-related documents such as product catalogs, manuals, and reports using natural language queries.The system uses **Retrieval Augmented Generation (RAG)** to retrieve relevant information from documents and generate accurate responses using a Large Language Model.

1.1 Scope of the Document

- Architecture of the chatbot system
- Document ingestion and processing pipeline
- Retrieval-Augmented Generation workflow
- Backend API design
- Data storage and access methods
- Security and performance considerations

1.2 Intended Audience

- Software Developers
- AI/ML Engineers
- System Architects
- Project Evaluators
- Faculty Members

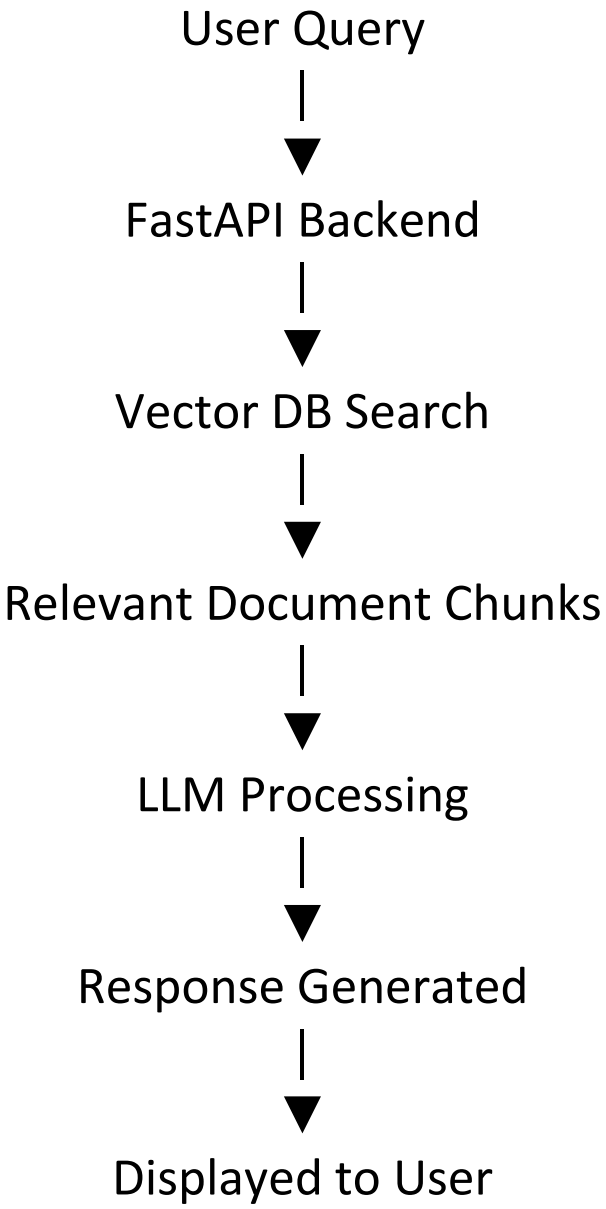
1.3 System Overview

The Retail Document Chatbot allows users to ask questions about retail documents. It uses Retrieval Augmented Generation (RAG) to retrieve relevant information from documents stored in a vector database before generating answers using a Large Language Model.

2.1 Application Design

```
+---------------------+
|   User Interface    |
|     Streamlit       |
+---------------------+
          |
          ▼
+---------------------+
|     Backend API     |
|      FastAPI        |
+---------------------+
          |
          ▼
+---------------------+
|    RAG Pipeline     |
|  LangChain Engine   |
+---------------------+
          |
          ▼
+---------------------+
| Vector DB + LLM     |
| Chroma + Groq       |
+---------------------+
```

2.2 Process Flow



2.3 Information Flow

Retail Documents

↓

Text Processing

↓

Vector Embeddings

↓

Vector Database

↓

Retriever

↓

LLM Model

↓

Generated Answer

↓

User Interface

2.4 Components Design

```
+-------------------+
| Document Loader   |
+-------------------+
        |
        ▼
+-------------------+
| Text Chunking     |
+-------------------+
        |
        ▼
+-------------------+
| Embedding Model   |
+-------------------+
        |
        ▼
+-------------------+
| Vector Database   |
+-------------------+
        |
        ▼
+-------------------+
| Retriever Module  |
+-------------------+
        |
        ▼
+-------------------+
| LLM Generator     |
+-------------------+
        |
        ▼
+-------------------+
| User Interface    |
```

2.5 Key Design Considerations

1. **Modular Architecture**
   The system is divided into separate layers including frontend, backend API, vector database, and AI model integration. This separation allows independent development and easier maintenance.
2. **Retrieval-Augmented Generation (RAG)**
   Instead of sending the entire document to the LLM, the system retrieves only the most relevant document chunks using embeddings and vector similarity search. This improves response accuracy and reduces token usage.
3. **Scalability**
   The architecture supports adding more documents, users, and embeddings without redesigning the system. Vector databases like FAISS can handle large-scale similarity searches efficiently.
4. **Performance Optimization**
   Document chunking and vector embeddings allow fast retrieval. Only relevant information is passed to the LLM to reduce latency.
5. **Security and Data Privacy**
   Uploaded documents remain within the system and are not permanently stored by external LLM providers. API keys are stored securely using environment variables.
6. **Ease of Use**
   The Streamlit interface allows non-technical users to upload documents and ask questions easily through a chat interface.

2.6 API Catalogue

1. Upload PDF API
Endpoint
POST /api/upload
Purpose
Uploads a retail PDF document to the system.
Processing Steps
1. Extract text from PDF
2. Split text into chunks
3. Generate embeddings
4. Store embeddings in vector database (e.g., **FAISS**)

Example Request
file: retail_report.pdf
Example Response

```
{
    "message": "Document uploaded successfully"
   }
```

2. Ask Question API
Endpoint
POST /api/ask
Purpose Allows the user to ask questions related to the uploaded documents.
Processing Steps
1. Convert question into embedding
2. Search relevant document chunks in vector database
3. Send retrieved context + question to **Groq** LLM
4. Generate contextual response

List Documents API

Endpoint

GET /api/documents

Purpose

Returns the list of uploaded documents.

Example Response

{
"documents": [
"retail_report.pdf",
"sales_analysis.pdf"

4. Query History API

Endpoint

GET /api/history

Purpose

Displays previously asked questions and answers.

Example Response

{
"question": "What is retail analytics?",
"answer": "Retail analytics refers to analyzing retail data to understand customer behavior."
}

3. Architecture Diagram (Conceptual)

```
User
  |
Streamlit UI
  |
FastAPI Backend
  |
RAG Pipeline
  |
Vector Database ---- Embeddings
  |
LLM
  |
Response to User
```

3.1 Data Access Mechanism

1. Backend API Layer (FastAPI)

- Document upload
- Query requests
- Interaction with vector database
- Communication with the LLM

2. Vector Database Access

FAISS performs similarity search using cosine similarity or Euclidean distance to find relevant chunks.

3. LLM API Access

Relevant chunks are sent to the LLM API for answer generation.

Workflow

1. User submits a query
2. Query is converted to embeddings
3. FAISS retrieves relevant document chunks
4. Context is sent to LLM
5. LLM generates the answer
6. Response returned to user

3.2 Data Retention Policies

1. Document Storage
Uploaded documents remain in the system until manually deleted or replaced.
2. Vector Embeddings
Embeddings remain stored in the vector database to allow fast retrieval for future queries.
3. Query Logs
User queries may be stored temporarily for debugging and system improvement.
4. Data Privacy
Sensitive documents should be protected and access restricted to authorized users.
5. Retention Duration
Organizations can define retention policies (e.g., 30 days, 90 days, or permanent storage).

3.4 Data Migration

Data migration ensures smooth transition when upgrading systems or changing storage mechanisms.

1. Vector Database Migration

Embeddings may be migrated from FAISS to scalable vector databases like Pinecone or ChromaDB.

2.Model Migration

The system may switch to newer embedding models or LLM providers.

3. Data Backup

Regular backups ensure no document or embedding loss during system upgrades.

4. Schema Updates

If the data model changes, migration scripts convert old formats into new formats.

3.5. Interfaces

The system provides several interfaces for interaction between components.

1. User Interface (Frontend)

Built using **Streamlit**, allowing users to:

- Upload retail documents
- Ask questions
- View AI-generated responses

2. Backend API Interface

Built using **FastAPI**, providing endpoints such as:

- /upload – Upload documents
- /query – Ask questions
- /process – Generate embeddings

3. LLM API Interface

External LLM services process queries using contextual document chunks.

4. Vector Database Interface

The backend interacts with FAISS to:

- Store embeddings
- Retrieve relevant chunks

4. RAG Pipeline Flow

```
Query
  |
Embedding
   |
Vector Search
    |
Retrieve Chunks
     |
Context Prompt
      |
LLM
       |
Answer
```

5. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system. Key aspects include:

- Security
- Performance
- Scalability
- Reliability
- Usability

6. Sequence Flow

User
|
UI
|
Backend
|
Vector DB
|
LLM
|
Backend
|
UI
|
User

7 References

Groq Documentation
https://console.groq.com/docs

LangChain Documentation
https://python.langchain.com

FAISS Documentation
https://faiss.ai

Streamlit Documentation
https://streamlit.io

Python Documentation
https://docs.python.org